MCP Server Design

Codebase Q&A and PR Review Assistant, Phase 2

Author	Todd Oppel
Repository	github.com/tpotechno/codebase-qa-assistant
Version	0.2.0 (Phase 2 complete)
Date	October 19, 2026
Status	Implemented, 17 automated tests passing

1. Summary

Phase 2 moves the assistant's tools out of the agent process and behind a custom **Model Context Protocol (MCP)** server served over Streamable HTTP. The server exposes three read-only tools: `search_code`, `read_file`, and a new documentation retrieval tool, `search_docs`. The agent no longer hard-codes its tools; it connects to the server as an MCP client, discovers the tools at runtime, and translates them into Claude Messages API tool definitions. Because the tools now live behind a standard protocol, the same server also works with Claude Code and any other MCP client without changes.

2. Why

In Phase 1, the tools were TypeScript functions compiled into the agent. That works for one app but creates three problems as the project grows:

- **No reuse.** Every new client (a web UI, Claude Code, a CI job) would have to re-implement or import the tools.
- **Tight coupling.** Adding a tool meant changing the agent's code and redeploying it.
- **No security boundary.** The tools ran with the agent's full permissions, with no separate place to enforce authentication or scope.

MCP addresses all three: it defines a standard contract for listing and calling tools, so the tool provider and the tool consumer can evolve and deploy independently. The goal was deliberate, not MCP for its own sake: a reusable, network-addressable tool contract is the foundation for the PR review tools in Phase 3 and the hosted demo in Phase 5.

Goals

- Serve the existing tools over MCP with no loss of behavior or safety.
- Add retrieval over the repository's documentation so the agent can answer intent and design questions, not only symbol lookups.
- Keep the agent loop unchanged apart from where it gets its tools.
- Make the server safe to run locally by default and safe to expose remotely when configured.

- Cover the client/server path with automated tests that need no API key.

Non-goals (this phase)

- Write operations (posting PR comments) — Phase 3.
- Embedding-based semantic search — deferred; see Section 6.
- A web user interface — Phase 4.
- Hosted deployment and OAuth — Phase 5.

3. What was built

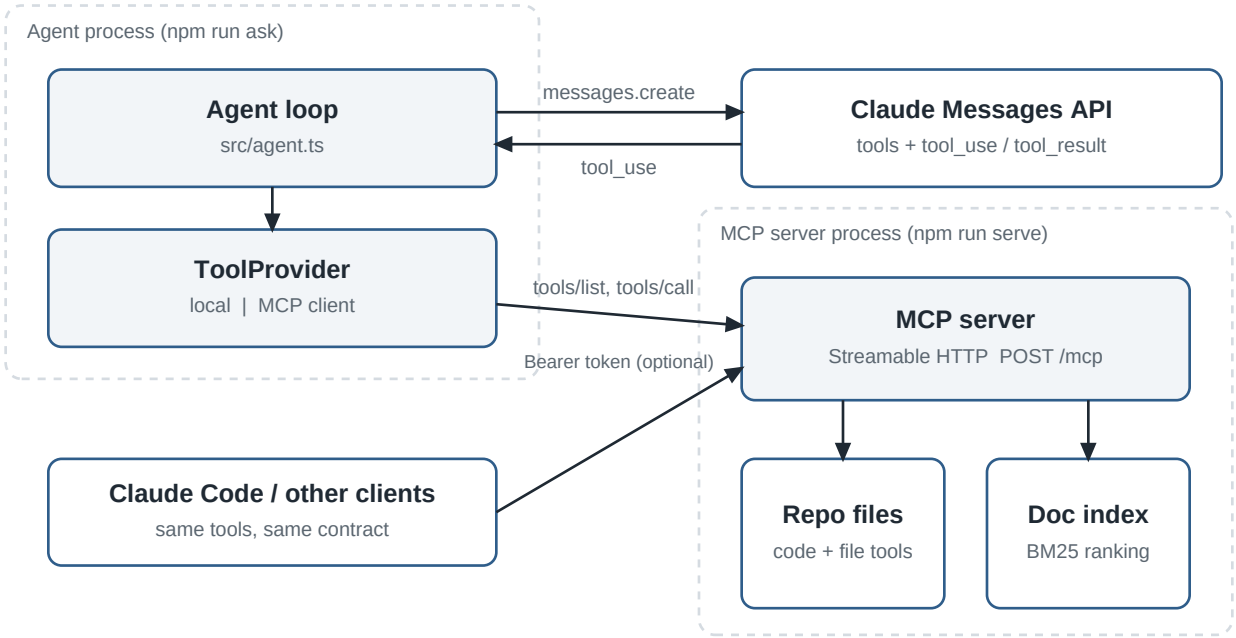

Figure 1. Runtime architecture. Dashed boxes are separate processes.

Components

File	Responsibility
`src/mcp-server.ts`	Builds the MCP server and registers the three tools with zod input schemas and read-only annotations. Serves it over Streamable HTTP at POST /mcp, with optional bearer-token auth and a GET /health endpoint.
`src/serve.ts`	Entry point for `npm run serve`. Reads configuration from .env and refuses to bind a non-local address without an auth token.
`src/docs.ts`	Documentation retrieval: walks the repo for Markdown and text docs, splits them into heading-scoped chunks, and ranks chunks with BM25.
`src/tool-provider.ts`	The `ToolProvider` interface plus two implementations: in-process tools (Phase 1 behavior) and an MCP client that discovers and calls tools on the server.

File	Responsibility
`src/tools.ts`	Tool implementations shared by both providers: code search, file reading with the path-traversal guard, and dispatch.
`src/agent.ts`	The hand-written Messages API loop, now taking a `ToolProvider` instead of a fixed tool list.

Tools

Tool	Input	Returns
`search_code`	query, k (1-25), path_glob	Up to k hits as file:line with one line of context on each side
`read_file`	path, start, end	Numbered lines, at most 200 per call, with a hint to page further; refuses paths outside the repo
`search_docs`	query, k (1-10)	Ranked documentation passages, each with file:line, heading path, and score

4. How it works

Request flow

A single question flows through the system as follows:

- **Connect.** The CLI reads `MCP_SERVER_URL` and opens an MCP client session to the server over Streamable HTTP (sending the bearer token if configured).
- **Discover.** The agent calls `tools/list`. Each MCP tool (name, description, JSON Schema) is mapped one-to-one to a Messages API tool definition.
- **Ask.** The agent sends the question, system prompt, and tool definitions to the Claude Messages API.
- **Execute.** When Claude responds with `tool_use` blocks, the agent forwards each to the server with `tools/call`. The server validates arguments against the zod schema, runs the tool confined to `REPO_PATH`, and returns text plus an `isError` flag.
- **Return.** The agent sends the results back as `tool_result` blocks (errors marked `is_error`) and repeats until Claude produces a final answer or the turn limit is reached.

Documentation retrieval (search_docs)

Documents are split at Markdown headings (ignoring heading-like lines inside fenced code blocks) and capped at about 1,500 characters per chunk. Each chunk records its file, starting line, and full heading path, such as "Quick start > Using the MCP server", so answers can cite exact locations. Chunks are ranked with **BM25**, the standard lexical relevance function behind most search engines, with the file name and headings weighted into each chunk's terms. The index is built on first use and cached for 30 seconds, so edits to docs show up without restarting the server.

Server design

- **Stateless Streamable HTTP.** Each POST creates a fresh server and transport and returns a JSON response. There are no sessions to track, so instances can run behind a load balancer. GET and DELETE on /mcp return 405.
- **Schema validation at the boundary.** Tool inputs are declared with zod. Invalid calls (for example an empty query) are rejected before any tool code runs.
- **Tool annotations.** All tools are marked `readOnlyHint` so clients can treat them as safe to call without confirmation.

5. Security

Risk	Mitigation
Reading files outside the repository	Every path is resolved against REPO_PATH and rejected if it escapes the root (relative or absolute). Covered by tests in-process and over HTTP.
Unauthorized access to the server	Optional bearer token checked with a constant-time comparison. Unauthenticated requests get HTTP 401.
Accidental public exposure	Binds to 127.0.0.1 by default. The server refuses to start on any non-local address unless MCP_AUTH_TOKEN is set.
DNS-rebinding attacks from a browser	The MCP SDK's Express app enables host-header validation automatically for local binds.
Oversized or binary content flooding the model	File-size caps, text-extension allowlist, skipped build and dependency folders, 200-line read windows, and capped result counts.
Secrets leaking into git	.env is git-ignored, and .env values override stale machine environment variables.

6. Key decisions and alternatives

Decision	Alternative considered	Why this choice
Client-side MCP: the agent connects with the MCP client SDK	Messages API MCP connector (mcp_servers parameter)	The connector requires a publicly reachable server. Client-side MCP works against localhost during development; the connector becomes an option once the server is hosted in Phase 5.
Streamable HTTP transport	stdio transport	stdio only works for a local child process. HTTP makes the server addressable by any client and deployable as a service.
Stateless request handling	Session-based server with stream resumption	These tools are simple request/response calls with no server-initiated messages, so sessions add complexity with no benefit, and statelessness scales horizontally.
BM25 lexical retrieval	Embeddings plus a vector store	No extra API keys, model downloads, or database; deterministic and testable. The tool's contract (query in, cited passages out) is the same, so embeddings can be added later without changing clients.

Decision	Alternative considered	Why this choice
Pluggable ToolProvider interface	Replace local tools with MCP outright	Keeps the Phase 1 path working with zero setup, makes the agent testable with either source, and new server tools reach the agent with no agent code change.
Bearer token auth	OAuth 2.1	Adequate for local and single-owner use. OAuth is the right fit for a multi-user hosted deployment and is planned for Phase 5.

7. Testing

The suite has 17 tests and runs in about two seconds with no API key, so it runs on every push in GitHub Actions. The MCP tests start a real server on a random port and connect to it with the real MCP client.

Area	What is verified
Tools (in-process)	Search hits and formatting, numbered line ranges, path-traversal refusal
Documentation retrieval	Heading-aware chunking, fenced-code handling, ranking returns the right README section, empty and unmatched queries
MCP server over HTTP	Tool discovery with JSON schemas, code and doc search, traversal refused with isError, schema validation rejects bad input, missing or wrong token rejected, /health
Agent loop	Tool round-trips, is_error flagging, turn limit, and an end-to-end run where the agent discovers tools from the MCP server and calls them via a scripted fake Claude client

8. Configuration and usage

Variable	Default	Purpose
`REPO_PATH`	current folder	Repository the tools are confined to
`MCP_SERVER_URL`	unset	When set, the agent uses the MCP server instead of in-process tools
`MCP_PORT`	3333	Server port
`MCP_HOST`	127.0.0.1	Bind address; non-local requires a token
`MCP_AUTH_TOKEN`	unset	If set, clients must send Authorization: Bearer <token>

```
# Terminal 1: start the server
npm run serve

# Terminal 2: with MCP_SERVER_URL=http://127.0.0.1:3333/mcp in .env
npm run ask -- "Why does the server refuse to bind to 0.0.0.0 without a token?"

# Use the same server from Claude Code
claude mcp add --transport http codebase-qa http://127.0.0.1:3333/mcp
```

9. Limitations and next steps

- **Lexical search only.** BM25 misses synonyms and paraphrases. Next: optional embedding-based ranking behind the same tool.
- **Substring code search.** `search_code` scans files directly. For large repos, swap in ripgrep or a prebuilt index.
- **Single repository per server.** REPO_PATH is fixed at startup; multi-repo support would take a repo parameter with an allowlist.
- **Phase 3:** add `get_pr_diff` and `post_review_comment` tools backed by the GitHub API, and move orchestration to the Claude Agent SDK.
- **Phase 4:** Angular web UI with streaming answers and a visible trace of each tool call, replacing the command line as the main way to use the assistant.
- **Phase 5:** evaluation suite, cost limits, tracing, OAuth, and a hosted deployment that can also use the Messages API MCP connector.